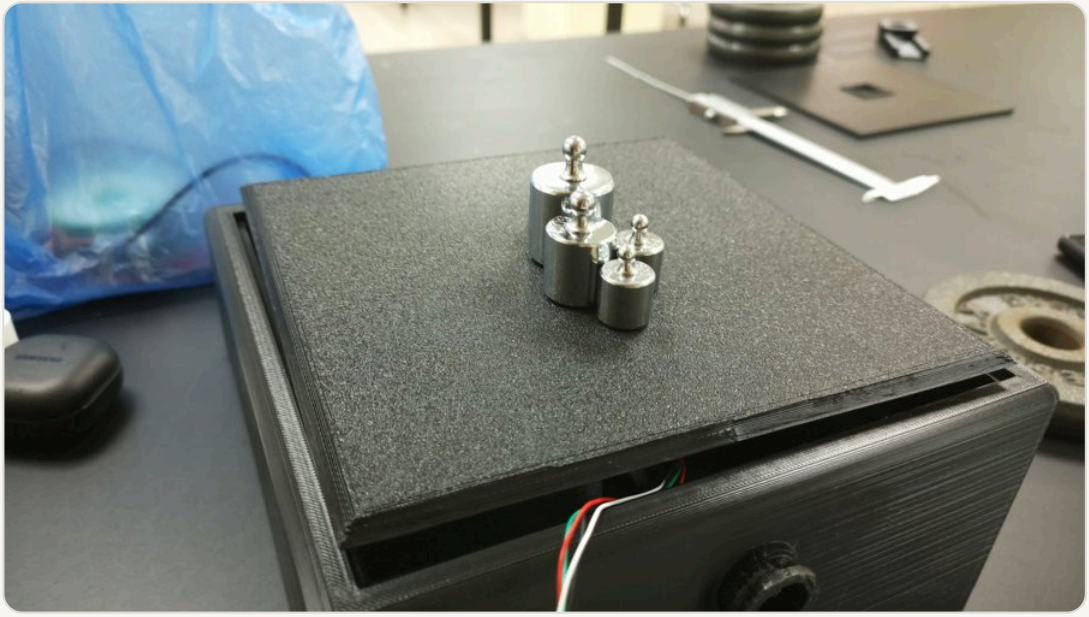
EE handover to the software team · 20 October 2026

Smart Food Scale Hardware Handover

What you get and what you build

Load cell ✓ Camera ✓ Touchscreen ✓ hardware.py ✓



What I am handing over

Everything the app needs from the hardware is ready on the Pi.

The hardware

Load cell, camera and touchscreen: set up and tested on the Pi.

hardware.py

The file your app uses to get the **weight** and the **photo**.

Handover PDF

Access, pins, rules and troubleshooting.

You **don't edit** hardware.py.

You **import** it and call it from your app:

```
import hardware as hw

grams = hw.get_weight()
```

Files on the Pi: /home/foodscale/ (hardware.py, scale.py, scale_calibration.json)

The 5 calls you need

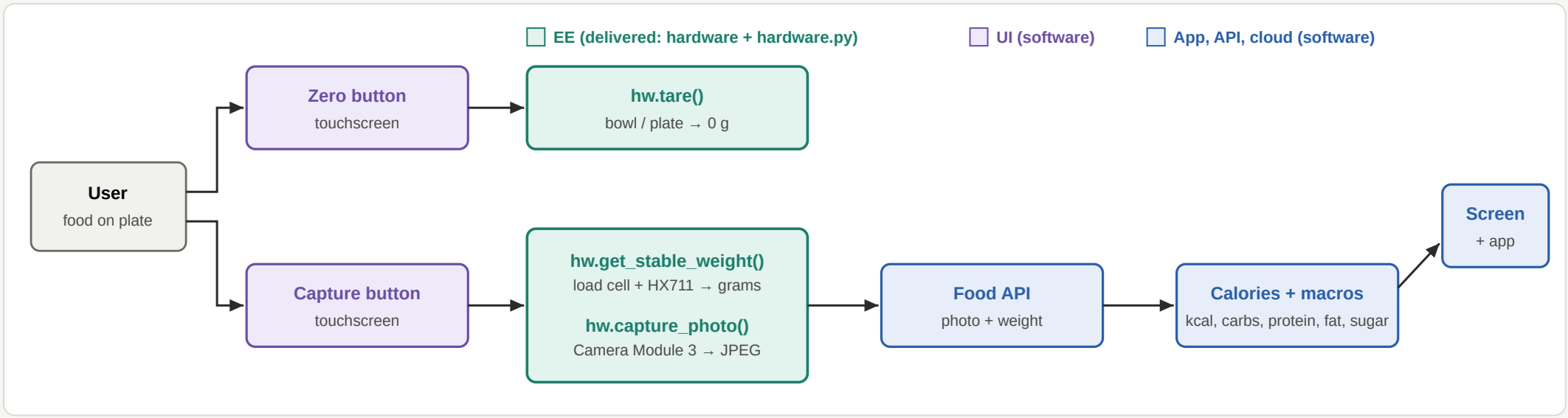
The app never touches pins, timing or calibration: these calls do it.

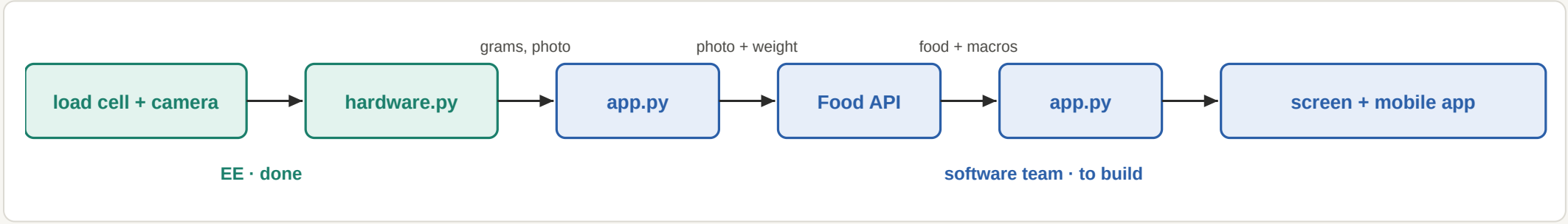
Call	What you get	When to use
hw.start()	—	app start-up, plate empty
hw.get_weight()	152.3 g	live weight on the screen
hw.tare()	—	**Zero** button
hw.get_stable_weight()	152.3 g	**Capture** button
hw.capture_photo()	photo file (JPEG)	**Capture** button

get_stable_weight() waits about 1 s until the reading is steady, so a tap while food is moving never sends a wrong weight.

How it all connects

Green is done (EE). Blue is what you build.





The load cell keeps reading in the background while the app waits for the API.

What you need to build, step by step

Six steps, in order. Each one builds on the one before.

1 app.py, the screen

Two buttons (**Zero**, **Capture**) and a live weight label.

Why: This is what the user touches, and it shows the result right away: calories, carbs, protein, fat and sugar.

2 Connect the buttons

Zero → `hw.tare()`

Capture → `hw.get_stable_weight()` + `hw.capture_photo()`

Why: This is how the hardware is used.

3 The API call

Send the **photo + weight** to the food API.

Why: The photo says what the food is; the weight says how much.

4 Show the result

Food name, calories, carbs, protein, fat and sugar on the screen. Show "Analysing…" while waiting.

Why: The call takes a few seconds.

5 Send to the mobile app

Save the meal record to the cloud.

Why: The user's history lives there.

6 Start on boot

The app opens by itself when the scale is switched on.

Why: So it works like a product.

Rules and next steps

Four rules keep the readings correct.

Start with the plate empty

`hw.start()` zeroes on what it sees.

One program uses the load cell

Stop any test script before starting the app.

Keep total load under 5 kg

Plate + bowl + food. `hw.is_overloaded()` warns at 4.5 kg.

Run the app with sudo

Gives the load-cell reader real-time priority.

Next for me (EE)

Battery runtime test with your app running: measured runtime vs the 2.8 h prediction, and no low-voltage warning.